Cursor Spend Dashboard — Project Reference

Complete inventory of every building block. Services, credentials, data, costs, and operations.

Open-source · MIT licensed

---

1. At a glance

Purpose	Internal visibility into Cursor AI spending, team and individual
Audience	Finance, engineering leadership, and the team being measured
Type	Read-only internal dashboard. Not customer-facing.
Data source	Cursor Admin API (one external dependency, nothing else)
Hosting	Google Cloud Run, `us-central1` (any region works)
GCP project	`YOUR_PROJECT_ID`
Refresh	Automatic, every 6 hours (00:00 / 06:00 / 12:00 / 18:00 IST)
Authentication	Google Workspace sign-in via Identity-Aware Proxy
Running cost	Effectively zero — see §9
LLMs at runtime	**None.** See §6

---

2. Technology stack

Layer	Choice	Why
Language	JavaScript (ES modules), Node.js 22+	No build step, no transpiler, no bundler. What you read is what runs.
Web server	Node's built-in `node:http`	Zero framework. ~120 lines.
Charts	Hand-written inline SVG, rendered server-side	No chart library, no CDN, no JavaScript required in the browser. A dashboard being presented to leadership should not depend on a script tag resolving.
Styling	Plain CSS with custom properties	Light and dark mode from one stylesheet.
Runtime deps	**One:** `@google-cloud/bigquery`	And even that is loaded lazily, so local development doesn't need it.
Container	`node:22-slim`	~15 line Dockerfile.
Tests	Custom offline harness	37 checks, no network needed.

Deliberately not used: React, Next.js, any chart library, any CSS framework, any ORM. For a page that renders numbers and bars, each of these would add build complexity and failure modes without adding capability.

---

3. Google Cloud services

Nine services. What each is for, and what happens if you remove it.

Service	Role here	Configuration	If it were removed
Cloud Run (Service)	Hosts the dashboard web page. Scales to zero when nobody is looking.	`cursor-spend-dashboard`, 1 GiB, min-instances 0, `--no-allow-unauthenticated`	No dashboard
Cloud Run (Job)	Runs the data collection, then exits. No inbound network path.	`cursor-spend-ingest`, 1 GiB, 20 min timeout, 2 retries	Data stops updating
Cloud Scheduler	Triggers the job every 6 hours	`0 */6 * * *`, timezone `Asia/Kolkata`	Manual refresh only
BigQuery	Stores all history	Dataset `cursor_usage`, partitioned by date	No history, no trends, no forecast
Secret Manager	Stores the Cursor Admin API key	Secret `cursor-admin-api-key`	Key would have to live in an env var or the image — much worse
Identity-Aware Proxy	Google sign-in gate in front of the dashboard	Enabled directly on Cloud Run, no load balancer	Dashboard would be public on the internet
Cloud Build	Turns source code into a container image on deploy	Invoked by `gcloud run deploy --source=.`	You'd build and push images by hand
Artifact Registry	Stores the built container images	Auto-created by Cloud Build	Nowhere to put images
IAM	Permissions for the service account and for people	Service account `cursor-spend@…`	Nothing could talk to anything

Cloud Logging is also in use, though nothing was configured for it — Cloud Run writes there automatically, and it's where you look when a job fails.

Service account

One identity, `cursor-spend@YOUR_PROJECT_ID.iam.gserviceaccount.com`, with the minimum roles needed:

Role	Needed for
`roles/bigquery.dataEditor`	Writing collected data
`roles/bigquery.jobUser`	Running queries
`roles/secretmanager.secretAccessor`	Reading the Cursor key (scoped to that one secret)
`roles/run.invoker`	Letting the scheduler start the job

It deliberately does **not** have project Owner or Editor.

---

4. Credentials and secrets

There is exactly **one** secret in this entire system.

The Cursor Team Admin API key

Format	`crsr_` followed by 64 hex characters
Scope	Read team members, spend and usage events
Created by	A Cursor team admin, from the team dashboard
Stored in	Google Secret Manager, secret name `cursor-admin-api-key`
Injected as	Environment variable `CURSOR_API_KEY`, at container start
Who can read it	Only the `cursor-spend` service account, plus GCP project admins
In source control?	**No.** `.gitignore` covers `.env.local`, `.env*`, `*.key`, `*.pem`, `service-account*.json`

> **A trap worth knowing about.** Cursor has two different things that both say "Admin." A **User API Key** whose *scope* reads `Admin` is for the Cursor Agent CLI and Cloud Agents — it returns `401 Invalid Team API Key` on every team endpoint. What this project needs is a **Team Admin API key**, which only a team admin can create. This cost us a false start; the error message does not make the distinction obvious.

Rotating it

```
printf '%s' 'crsr_NEWKEY' | gcloud secrets versions add cursor-admin-api-key --data-file=-
gcloud run jobs update cursor-spend-ingest --region=us-central1 \
  --set-secrets=CURSOR_API_KEY=cursor-admin-api-key:latest
```

No code change, no redeploy. Rotate whenever someone with access leaves.

What there isn't

No database passwords (BigQuery uses the service account identity). No user accounts or passwords (Google sign-in handles that). No session cookies we manage. No third-party API keys beyond Cursor.

---

5. Data storage

Where it lives

BigQuery dataset `cursor_usage` in the `US` multi-region. Three tables.

`usage_events` — one row per AI call. Partitioned by date, clustered by user and model.

Column	Type	Meaning
`tsMs`	INT64	When the call happened (epoch ms)
`istDate`	DATE	The IST calendar day — what all charts group by
`userEmail`	STRING	Who made the call
`model`	STRING	e.g. `composer-2.5-fast`, `Opus 5 (Auto Balanced)`
`kind`	STRING	`Usage-based` or `Included in Business`
`isOnDemand`	BOOL	True when this call bills on top of the subscription
`onDemandCents`	FLOAT64	The billed portion
`includedCents`	FLOAT64	The portion the pool absorbed
`chargedCents`	FLOAT64	Total value of the call
`inputTokens` … `cacheReadTokens`	INT64	Token counts
`cloudAgentId`, `automationId`, `serviceAccountId`	STRING	Set when a machine, not a person, made the call
`isHeadless`, `isNonHuman`	BOOL	Automated usage flags
`conversationId`	STRING	Groups calls in one session

`spend_snapshots` — the official Cursor figure, captured at each run. This table exists because Cursor's spend endpoint has **no history**. Each row is a moment in time that cannot be recovered later.

`members` — the team roster, so departed staff still resolve to a name in historical data rather than becoming an anonymous email.

Volume

Events per month	~8,000
Full history (Oct 2025 → now)	~35,000 rows
Storage	Well under 100 MB
BigQuery free tier	10 GB storage, 1 TB queries/month

We are using roughly **1%** of the free tier. This will not generate a bill.

Retention

Ours is indefinite — the whole point is history Cursor won't keep. Cursor's own retention reaches back to about **October 2025**; a 365-day-ago probe returned nothing, September 2025 returned zero events, October returned 944.

How re-runs stay safe

Usage events carry **no unique ID**, so there is no reliable way to detect a duplicate row. Instead, ingestion replaces **whole days**: re-collecting a day deletes everything for that day and writes the fresh set.

This has a second benefit. Cursor revises recent data for a day or two, and a strictly "append what's new" design would bake provisional numbers in forever. Whole-day replacement means corrections apply themselves.

Local mode

For development the same code writes JSON files to `./data` instead of BigQuery. Identical interface, identical results, no cloud account needed. Switch with `STORAGE_DRIVER=file` or `STORAGE_DRIVER=bigquery`.

---

6. AI and LLM usage

The running system uses no AI whatsoever. Not one call to a language model. It fetches numbers from an API, adds them up, and draws bars.

This matters for three reasons: there is no per-query cost, no non-determinism — the same data always produces the same page — and no data leaving Google Cloud except the read-only call to Cursor.

The distinction worth being clear about: **Claude was used to build this project** — to explore the API, write the code and produce these documents. That was a one-off development cost. The deployed system contains no AI, no model weights, and no API keys to any AI provider other than Cursor's read-only reporting key.

The *subject* of the dashboard is AI spending. The dashboard itself is arithmetic.

---

7. Data flow in detail

Every 6 hours:

1. Cloud Scheduler fires an authenticated HTTP call to the Cloud Run Jobs API.
2. Cloud Run starts a container from the stored image.
3. The container reads `CURSOR_API_KEY`, injected from Secret Manager at start.
4. `POST /teams/spend` → current-cycle figures, plus `subscriptionCycleStart`, which is where the true billing-cycle boundary comes from.
5. Snapshot saved to `spend_snapshots`.
6. `GET /teams/members` → roster, saved to `members`.
7. `POST /teams/filtered-usage-events` → every call since the later of *four days ago* or *the cycle start*, paginated 1,000 at a time.
8. Each event is normalised (see §11) and written to `usage_events`, replacing whole days.
9. A completion marker is written. The container exits. Billing stops.

Typical run: **under two minutes.**

When someone opens the page:

1. Request hits Identity-Aware Proxy.
2. Not signed in → Google sign-in. Signed in but not authorised → 403.
3. Authorised → request reaches Cloud Run, which may cold-start (1–3 seconds).
4. The server queries BigQuery for the current cycle and the last 90 days.
5. It computes every figure — totals, per person, per model, forecasts, alerts.
6. It cross-checks its own arithmetic against the latest Cursor snapshot and records the difference.
7. It renders HTML with inline SVG charts and returns it.
8. Results are cached for 60 seconds, so a room full of people refreshing at once causes one query, not thirty.

---

8. Security and access

Getting in

Identity-Aware Proxy sits in front of the dashboard. There is no application login, no shared password, no session management of our own. Access is granted per person, per group, or per domain:

```
./deploy/grant.sh YOUR_PROJECT_ID group:cursor-dashboard@yourcompany.com
```

A Google group is strongly preferable to a whole domain. Per-person AI spend is not something everyone at the company needs, and a group is one place to change when someone joins or

leaves.

Blast radius

Risk	Mitigation
Dashboard is compromised	It has read-only BigQuery access and no Cursor key. Nothing to steal, nothing to break.
Collector is compromised	It has no inbound network path at all. Nothing external can reach it.
API key leaks	Read-only against Cursor reporting. Rotate in one command.
Someone leaves	Remove from the Google group. Access ends immediately.
Wrong numbers shown	The page reconciles itself on every load and flags discrepancies in the footer.

Personal data

The dashboard displays employee names, emails and individual spending. That is its purpose, but it should be treated as sensitive. Two consequences:

- Grant access by group, not domain.
- Test fixtures in the repository use **fictional** identities (`dev-01@example.com`). Real colleagues' names and spend rankings are not committed to source control.

---

9. Cost

Service	Usage	Monthly
Cloud Run Service	Only while someone is viewing	~$0 (free tier)
Cloud Run Job	4 runs/day × ~2 min	~$0 (free tier)
Cloud Scheduler	1 job	Free (3 free/month)
BigQuery storage	<100 MB	Free (10 GB free)
BigQuery queries	Small, cached 60s	Free (1 TB free)
Secret Manager	1 secret	~$0.06
Artifact Registry	~200 MB image	~$0.02
Total		**Under $1/month, likely $0**

The one thing that could change this is `--min-instances=1`, which keeps a container warm to avoid cold starts. That's roughly **$12–15/month**. Worth enabling before a live demo; worth turning off afterwards.

---

10. Repository and operations

Repository

`github.com/kgrao22/cursor-usage-dashboard` — MIT licensed.

Clone it, configure `.env.local`, and deploy into your own GCP project. Nothing in the repository is tied to a particular organisation: budgets, project ID, region and retention window are all configuration.

Layout

```
src/
  config.js      budgets, thresholds, model grouping
  time.js        IST handling, billing cycle, forecasting
  cursor.js      Cursor API client — every quirk lives here
  store.js       storage: file (local) or BigQuery (cloud)
  aggregate.js   every displayed number, computed once
  render.js      server-rendered HTML and SVG charts
  server.js      read-only HTTP server
scripts/
  backfill.mjs   one-time history load
  ingest-cli.mjs routine refresh — what the Cloud Run Job runs
  smoke-test.mjs 26 offline checks
  store-test.mjs 11 storage checks
  probe*.mjs     the API exploration this was built from
deploy/
  setup.sh       one-time GCP setup
  deploy.sh      repeatable deploy
  grant.sh       grant a person or group access
docs/
  ARCHITECTURE.md, PROJECT-REFERENCE.md
```

Common operations

```
# Run locally
npm install && npm start

# Refresh data now, without waiting for the schedule
gcloud run jobs execute cursor-spend-ingest --region=us-central1 --wait

# Deploy a change
./deploy/deploy.sh YOUR_PROJECT_ID

# Give someone access
./deploy/grant.sh YOUR_PROJECT_ID user:name@yourcompany.com

# See why a run failed
gcloud run jobs executions list --job=cursor-spend-ingest --region=us-central1

# Change the budgets without touching code
gcloud run services update cursor-spend-dashboard --region=us-central1 \
  --set-env-vars=ON_DEMAND_BUDGET_USD=2000
```

Configuration

Variable	Default	Purpose
`MONTHLY_BUDGET_USD`	3000	Total budget
`ON_DEMAND_BUDGET_USD`	1500	On-demand cap
`WARN_AT_PROJECTED`	0.9	Warn at 90% of projected budget
`CONCENTRATION_ALERT`	0.5	Flag one person exceeding this share
`SPIKE_MULTIPLE`	2.5	Flag a day this many times the trailing mean
`EXPENSIVE_CALL_USD`	2	Individual call cost worth naming
`STORAGE_DRIVER`	`file`	`file` or `bigquery`

---

11. Cursor API — documented vs actual

Everything below was verified against the live team. Cursor's published documentation disagrees with its own API in several places, and each disagreement would have produced

wrong numbers.

Reality	Consequence
`overallSpendCents` is documented but **never returned**	Never referenced
`includedSpendCents`, `billingTier`, `autoPercentUsed`, `apiPercentUsed`, `totalPercentUsed` are returned but **undocumented**	Used — they populate only for tiered self-serve teams (`billingTier`, e.g. `TIER_1000`)
`timestamp` is epoch ms **as a string**	Standard date parsing throws on it
`serviceAccountId` is the literal string `"null"` when unset	It is therefore *truthy* — a naive check marks **every** call as a service account
`chargedCents` ≠ `tokenUsage.totalCents + cursorTokenFee` (held for 953 of 1,208 events, broke for 255)	`chargedCents` is treated as authoritative and never recomputed
Billing cycle starts at **15:30 IST**, not midnight	Anchored on `subscriptionCycleStart`; assuming midnight pulls ~15.5 hours of the previous cycle into every total
`/teams/spend` has no history and no date parameters	Snapshotted every run
Billing groups return `400 — not enabled`	No team-level rollups available
Rate limits: 60/min on usage events, 20/min elsewhere	Client throttles with 15% headroom, backs off on 429

Verification

```
npm test
```

37 checks, no network required: normalisation, IST boundary handling, cycle anchoring, conservation (every breakdown must re-sum to the total), storage idempotency, and empty-state rendering. Both bugs found during development are pinned as permanent fixtures.

---

12. Known limitations

Limitation	Impact	Possible fix
No Slack or email alerts	Warnings only appear if someone opens the page	Add a webhook to the ingest job
No team or department grouping	Cursor billing groups aren't enabled for this team	Enable in Cursor, or add a mapping file
Read-only	Cannot set spend limits from here	Enterprise-plan API only
Single region	One region only	Fine — an internal dashboard needs no failover
Cursor revises recent data	Today and yesterday can move	Already handled by whole-day replacement
BigQuery driver untested locally	Cloud path proven only on first deploy	Verified during deployment

Reasonable next steps

1. **Slack alerts** — the biggest gain. Push a warning when projected spend crosses budget, so nobody needs to remember to look.
2. **Per-user limits in Cursor** — none are currently set; `monthlyLimitDollars` is `null` for everyone. Nothing caps an individual except the team total.
3. **Cost per accepted line of code** — the productivity data is already available from the API; only the dashboard tile is missing.
4. **Multi-tool view** — the same pattern would cover Gemini API and GCP spend, which is where the original $50k surprise came from.